Next.js Deployment Guide (Ubuntu)

Windows development → Ubuntu server with Nginx → http://192.168.6.128/ocr

Setup: Next.js app using static export (`output: "export"`), server running Ubuntu (22.04 / 24.04, also valid for Debian) with Nginx, files copied with FileZilla. No Node.js is needed on the server; Nginx serves the built `out` folder.

What changes from RHEL: Nginx runs as `www-data` (not `nginx`); there is no SELinux, so `restorecon` is dropped; the firewall is `ufw` (not `firewall-cmd`); site configs go in `sites-available` / `sites-enabled`, and the default site must be disabled.

Reference: next.config.ts

```
import type { NextConfig } from "next";

// Set when the app is hosted under a sub-path, e.g. BASE_PATH=/ocr.
const basePath = process.env.BASE_PATH?.replace(/\/+$/, "") || undefined;

const nextConfig: NextConfig = {
  output: "export",      // npm run build writes static files to ./out
  basePath,
  trailingSlash: true,   // emits /history/index.html etc.
  images: { unoptimized: true },
  reactStrictMode: true,
};

export default nextConfig;
```

No change to this file is needed. The sub-path comes from the BASE_PATH environment variable at build time.

Step 0: Install Nginx (if not installed yet)

```
sudo apt update
sudo apt install -y nginx
sudo systemctl enable --now nginx
```

Step 1: Build and check the out folder (Windows)

Build with BASE_PATH set (PowerShell):

```
cd C:\path\to\your\project
$env:BASE_PATH="/ocr"
npm run build
```

Command Prompt (cmd) alternative (no space before &&):

```
set BASE_PATH=/ocr&& npm run build
```

Optional: create `.env.production` in the project root with the line below, so a plain `npm run build` always targets /ocr (`npm run dev` is unaffected):

```
BASE_PATH=/ocr
```

Check: open `out\index.html` in Notepad and search for `_next`.

- `/ocr/_next/...` → correct, continue.
- `/_next/...` → BASE_PATH was not applied; rebuild.

basePath does not fix these automatically:

- Files from `public/`: `src="/logo.png"` must become `/ocr/logo.png`.

- Backend API URL: put it in a `NEXT_PUBLIC_...` variable in `.env.production` (baked in at build time; must not be localhost).

Step 2: Upload with FileZilla

1. Right panel (server): go to your home folder, e.g. `/home/youruser`.
2. Right-click → Create directory → name it `ocr`, then open it.
3. Left panel (Windows): open your project's `out` folder.
4. Select everything inside `out` (Ctrl+A) and drag it to the right panel.

Result: `/home/youruser/ocr/index.html` and `/home/youruser/ocr/_next/` (not `.../ocr/out/...`).

Step 3: Move files into the web folder (SSH)

```
sudo mkdir -p /var/www/html/ocr
sudo rm -rf /var/www/html/ocr/*
sudo cp -r ~/ocr/* /var/www/html/ocr/
sudo chown -R www-data:www-data /var/www/html/ocr
sudo chmod -R u=rwX,go=rX /var/www/html/ocr
```

Ubuntu has no SELinux, so there is no `restorecon` step. The `chmod` makes all folders readable/enterable and all files readable by Nginx (755 for folders, 644 for files).

Step 4: Create the Nginx site config

```
sudo nano /etc/nginx/sites-available/ocr
```

In nano: paste the block, press **Ctrl+O**, **Enter** to save, then **Ctrl+X** to exit. (`sudo vi` works too: `i` to insert, `Esc`, `:wq`.)

```
server {
    listen 80;
    server_name 192.168.6.128;

    root /var/www/html;
    index index.html;

    location = /ocr {
        return 301 /ocr/;
    }

    location /ocr/ {
        try_files $uri $uri/ $uri.html =404;
        error_page 404 /ocr/404.html;
    }

    location /ocr/_next/static/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Enable the site, disable Ubuntu's default site, test and reload:

```
sudo ln -s /etc/nginx/sites-available/ocr /etc/nginx/sites-enabled/ocr
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl enable --now nginx
sudo systemctl reload nginx
```

Removing the `sites-enabled/default` link only disables the default site; the original stays in `sites-available/default`. Re-enable it any time with `sudo ln -s /etc/nginx/sites-available/default /etc/nginx/sites-enabled/`.

Step 5: Open the firewall (ufw)

Check whether ufw is active first:

```
sudo ufw status
```

If it says **inactive**, port 80 is already open and nothing more is needed. If it is **active**:

```
sudo ufw allow 'Nginx HTTP'
sudo ufw reload
```

If you decide to turn ufw on (`sudo ufw enable`), run `sudo ufw allow OpenSSH` first, or you will lock yourself out of SSH and FileZilla (SFTP).

Step 6: Test

Open `http://192.168.6.128/ocr` in a browser. Also open a sub-page directly (e.g. `http://192.168.6.128/ocr/history/`) and refresh it.

Updating the app later

1. Windows: `npm run build` with `BASE_PATH=/ocr`.
2. FileZilla: upload the contents of `out` to `~/ocr`, choose **Overwrite**.
3. Linux:

```
sudo rm -rf /var/www/html/ocr/*
sudo cp -r ~/ocr/* /var/www/html/ocr/
sudo chown -R www-data:www-data /var/www/html/ocr
sudo chmod -R u=rwX,go=rX /var/www/html/ocr
```

No Nginx reload is needed for content updates.

Troubleshooting

Symptom	Fix
403 Forbidden	Check ownership and permissions: `sudo chown -R www-data:www-data /var/www/html/ocr` and `sudo chmod -R u=rwX,go=rX /var/www/html/ocr`. Confirm with `namei -l /var/www/html/ocr/index.html`.
Default “Welcome to nginx” page	Use the URL with /ocr; confirm `/etc/nginx/sites-enabled/default` is removed and `sites-enabled/ocr` exists; `sudo nginx -t` shows no errors; reload Nginx.
Unstyled page, /_next/... 404 in browser console	Built without `BASE_PATH=/ocr`. Rebuild and check `out\index.html`.
Images missing	Image src paths need the `/ocr` prefix.
Dynamic route like /doc/[id] gives 404	Static export only builds paths from `generateStaticParams`. Use a query string (`/doc/?id=5`) for runtime IDs.
API calls fail / CORS errors	Wrong `NEXT_PUBLIC_` backend URL (rebuild after changing), or backend must allow origin `http://192.168.6.128`.
Page does not load at all	Firewall (Step 5, `sudo ufw status`); `sudo systemctl status nginx`.

Symptom	Fix
nginx -t: duplicate default server	Another file in `sites-enabled/` or `conf.d/` also listens on port 80 as default; remove the extra one.

Useful commands:

```
sudo tail -20 /var/log/nginx/error.log
sudo systemctl status nginx
sudo nginx -t
ls -l /var/www/html/ocr          # expect owner www-data, files -rw-r--r--
namei -l /var/www/html/ocr/index.html   # checks every folder on the path
sudo ufw status
```

RHEL → Ubuntu command map

RHEL	Ubuntu
`nginx:nginx (owner)`	`www-data:www-data`
`sudo restorecon -Rv /var/www/html/ocr`	`not needed (no SELinux); use chmod -R u=rwX,go=rX`
`/etc/nginx/conf.d/ocr.conf`	`/etc/nginx/sites-available/ocr + symlink in sites-enabled`
`sudo firewall-cmd --permanent --add-service=http`	`sudo ufw allow 'Nginx HTTP'`
`sudo firewall-cmd --reload`	`sudo ufw reload`
`ls -Z (SELinux labels)`	`ls -l / namei -l (permissions)`
`sudo dnf install nginx`	`sudo apt install nginx`